

A deep learning dismantling approach for understanding the structural vulnerability of complex networks

Li Hong^a, Mengqiao Xu^{b,*}, Yu Liu^a, Xuemei Zhang^a, Changjun Fan^c

^a School of Software Technology, Dalian University of Technology, No. 321 Tuqiang Street, Economic and Technological Development Zone, Dalian City, Liaoning Province 116620, China

^b School of Economics and Management, Dalian University of Technology, No.2 Linggong Road, Ganjingzi District, Dalian City, Liaoning Province, 116024, China

^c College of Systems Engineering, National University of Defense Technology, Changsha Province 410073, China

ARTICLE INFO

Keywords:

Complex networks
Structural vulnerability
Network dismantling problem
Graph attention network
Unsupervised learning

ABSTRACT

Understanding the structural vulnerability of real-world complex networks is a crucial issue. This issue can be addressed by solving the so-called network dismantling problem, which aims to identify the smallest set of nodes for the fastest dismantling of a network. The network dismantling problem is a computationally challenging task due to its NP-hard nature. The powerful learning capabilities of machine learning methods provide a new perspective for solving this problem. Most existing deep learning methods are based on supervised learning, which rely on pre-labeled training data. These existing methods have achieved promising results in completely dismantling a network. However, they may not perform very well in identifying a very small number of nodes whose removal would significantly degrade—though far from fully dismantle—the network performance. This issue is particularly relevant to many real-world networks like transportation and infrastructure networks. This paper proposes a novel unsupervised learning model called GASC (Graph Attention network integrated with Shortcut Connections mechanism), to address this issue regarding the network dismantling problem. The GASC model employs the graph attention network to learn the underlying features of a network's topological structure and integrates the shortcut connections mechanism to overcome the vanishing gradients problem. Besides, we incorporate a network tailoring method to further improve the performance of our model. We performed comprehensive experiments on both synthetic networks and various real-world networks. The results validate that the proposed approach outperforms existing state-of-the-art methods and demonstrates significant robustness.

1. Introduction

Most real-world systems like road transportation networks [1], global liner shipping networks [2], aviation networks [3], social networks [4], etc., are modeled as complex networks. These systems can use graph theory methods to analyze networks. Each network is expressed as a graph $G = (V, E)$, which consists of the node set V and the edge set E . When a small number of nodes or edges in a network experience failures or malicious attacks, the structure and functionality of this network may be severely disrupted, potentially leading to collapse [5–7]. This highlights the vulnerability of the network's structure and function. A typical example was the massive power outage that occurred in North America in August 2003, which caused widespread blackouts across dozens of provinces in the region [8]. Therefore, understanding structural

vulnerability of complex networks is a significantly important problem [9–11].

Extensive research has been conducted on the characteristics of network vulnerability across various failure scenarios [12,13]. The study in Ref. [14] explores the vulnerability of complex networks by analyzing percolation on random graphs, providing insights into how the structural characteristics of networks affect network vulnerability to failures. Valuable findings have derived from investigating network failure phenomena, which encompass cascading failure, phase transition, and network percolation [15]. Removing a few nodes in the network could lead to the functional failure of their closely connected nodes, which may cause local structural damage and even culminate in network collapse. This observation has inspired extensive research focused on attacking the smallest set of nodes to quickly disintegrate the

* Corresponding author.

E-mail address: stephanie@dlut.edu.cn (M. Xu).

<https://doi.org/10.1016/j.chaos.2025.116148>

Received 14 June 2024; Received in revised form 31 December 2024; Accepted 11 February 2025

Available online 19 February 2025

0960-0779/© 2025 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

network, which is the so-called “Network Dismantling (ND) problem” in complex network science [16,17].

The ND problem belongs to the discrete combinatorial optimization problem, which has been proved to have the NP-hard nature [16]. Therefore, finding and removing the smallest set of nodes to quickly dismantle the network is a significant and challenging task. Some approaches have been proposed for solving the ND problem [18,19]. Classical methods are mainly mathematical programming methods, centrality methods and heuristic methods. Mathematical programming methods have dynamic programming [20], branch-and-bound [21], etc. However, mathematical programming methods may become helpless with the problem size increasing. Thus, some researchers focus on adopting centrality methods, mainly including degree centrality [22,23], K-core [24], betweenness centrality [25], etc. A dismantling strategy is devised by sequentially removing nodes or edges in descending order of their centrality index when dealing with a large-size complex network. Although the methods based on centrality index are straightforward and readily implementable, the collapse performance is not ideal in many cases. To enhance the efficacy of network dismantling some researchers have proposed numerous complex network dismantling methods based on heuristic methods such as greedy algorithm [26], artificial bee colony algorithm [27], genetic algorithm [28], etc. In general, the heuristic methods may be able to obtain better network dismantling schemes, but they have higher time complexity.

The number of nodes in real-world complex systems is increasing. The computational complexity of algorithms may become a bottleneck restricting the research on the ND problem. Proposing high-efficiency and low-complexity disintegration algorithms is a priority on the ND problem in the current research. Deep neural networks provide a potent instrument for computing and optimizing complex networks. Deep learning methods and deep reinforcement learning have been successfully applied to analyze the vulnerability of networks [29,30]. Therefore, employing deep learning methods may offer a promising approach to solving the ND problem.

Recently, the studies [31–35] have utilized deep reinforcement learning [36] and deep neural networks [37] to address the ND problem. For instance, Fan et al. [31] proposes the FINDER method, which adopts graph representation learning for encoding the states and actions. This method integrates deep neural models and Q-learning to optimize the objective function. While these approaches demonstrate effective performance in solving the ND problem and exhibit some degree of generalization, they are hindered by their relatively complex algorithm models, which can make convergence challenging and lead to long training times. The studies [38–41] have designed deep neural models for solving the ND problem and have utilized supervised learning to train these models. Grassia et al. [38,39] proposes the GDM method and CoreGDM method respectively to address the ND problem. The GDM method [38] uses graph attention networks to obtain novel insights into the inherent features of topological structures within complex networks. The CoreGDM method [39] is a further development of the GDM method, integrating it with the CoreHD algorithm. Both methods employ supervised learning to train models on a large dataset comprising numerous small synthetic networks. Each synthetic network undergoes optimal dismantling through brute-force with nodes receiving numeric labels as the learning targets. Following training, the models are then applied to generalize their performance to real-world networks. Zhang and Wang [40] design a novel neural network model named NIRM to solve the ND problem by learning both local structure and global structure for the nodes. The NIRM model is trained by supervised learning and introduces a score propagation mechanism to assign labels to some small synthetic networks. Liu and Wang [41] propose a NEES method based on self-supervised learning to solve the ND problem. The NEES method utilizes self-attention techniques to grasp the hierarchical importance of nodes across different scales, enabling the extraction of fundamental structures. In summary, the machine learning approaches presented in these articles have

demonstrated effective performance in tackling the ND problem. Inspired by these successful studies, we will further explore the promise of machine learning-based approaches to tackle the ND problem.

Supervised learning is commonly used to train models through specific downstream tasks with manual labels. Unfortunately, the ND problem have the NP-hard nature, which indicates that finding the optimal solution is very difficult even on the small network [41]. Therefore, obtaining the so-called true ground labels is a challenging task. Unsupervised learning [42] can uncover patterns and rules of network dismantling from data without relying on pre-labeled training data. This approach may be closer to real-world scenarios and can better adapt to different types and scales of networks. Very few studies employ unsupervised learning methods to address the network dismantling problem in machine learning-based approaches. The recently proposed DCRS method [43] effectively addresses the ND problem using unsupervised learning. The DCRS model exhibits a relatively high time complexity due to its incorporation of the ROIX algorithm [44] for assessing node topological significance in dismantling networks. The ROIX algorithm classifies the node into similar regional structural features and employs the graph convolutional network (GCN) [45] to further distinguish and learn the node characteristics. As the number of network nodes increases, the calculation time of the ROIX algorithm extends. These machine learning-based methods have achieved promising results when the network is completely dismantled. However, they may not perform very well in identifying a very small number of nodes whose removal would significantly degrade—though far from fully dismantle—the network performance. This issue is especially pertinent in various real-world networks such as transportation and infrastructure systems. Additionally, most of these machine learning-based network dismantling methods adopt supervised learning or deep reinforcement learning [46]. However, supervised learning methods face challenges related to labeling and deep reinforcement learning methods often struggle with training convergence. Moreover, these methods often fail to fully explore the influence of multi-hop neighbors on nodes within the network.

To fill the aforementioned gaps in current research within this field, this paper develops a new deep-learning model utilizing unsupervised learning for solving this issue regarding the ND problem. Our model, called GASC, uses the Graph Attention networks [47] to learn the underlying features of a network's topological structure and integrates the Shortcut Connections mechanism [48] to overcome the vanishing gradients problem and enhance the network's representation and learning capabilities. Our GASC model can effectively obtain the high-level features of the nodes by aggregating the features of each node and its multi-hop neighbors, with the aim of deriving the dismantling score for each node. Then, an unsupervised learning approach considering cascading effects is designed to train the GASC model, which can not only handle the difficult problem of labeling in supervised learning, but also can effectively guide the model to learn the dismantling score for each node. Our model can be trained on small synthetic networks but can generalize well to various real-world networks. The experimental results validate the effectiveness and efficiency of the proposed approach. Furthermore, to further improve the performance of our approach, inspired by the FINDER method [31], our model incorporates a network tailoring method. The network tailoring method employs the configuration model (CM) [49] to generate the training networks that preserves important characteristics of the target network. We performed comprehensive experiments on synthetic networks and various real-world networks, validating that our approach outperforms existing state-of-the-art methods and exhibits strong robustness.

The subsequent sections of this paper unfold as follows. Section 2 provides the preliminary of the network dismantling problem. Section 3 presents the methodology of our approach. Section 4 shows the main experiments and analyses of the results. Section 5 serves as the concluding section, discussing both the strengths and limitations of the proposed approach, and providing a discussion on potential directions

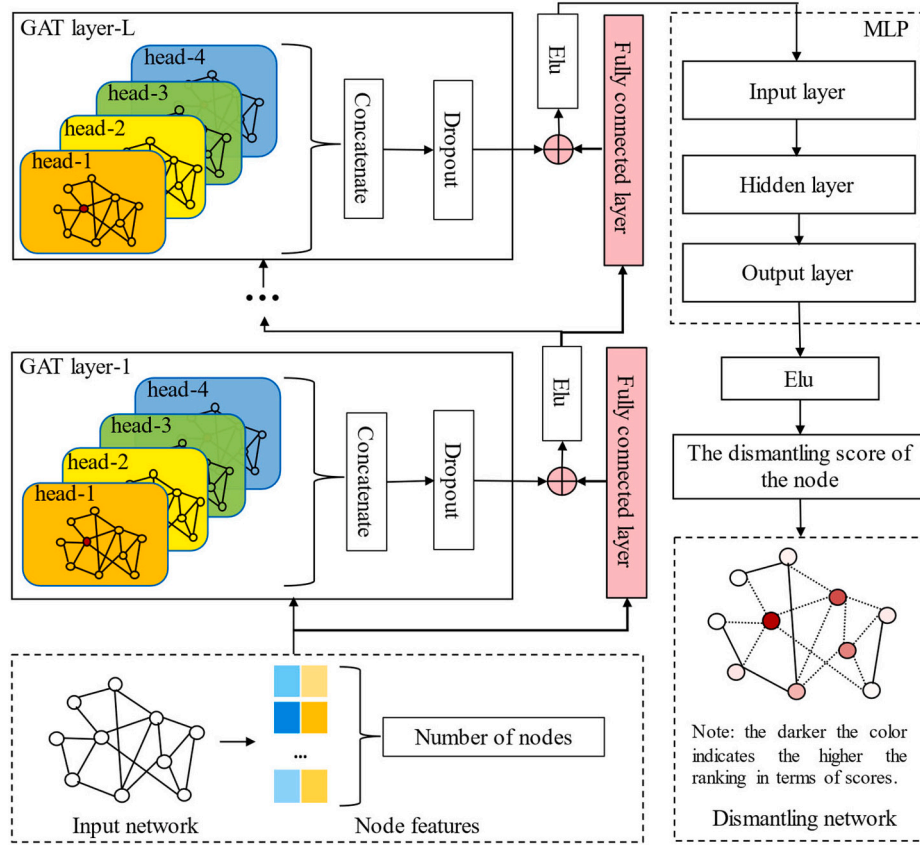


Fig. 1. The proposed GASC model structure for the network dismantling problem.

for further research.

2. Preliminary

The network dismantling problem involves identifying and removing the smallest set of nodes in the network to disintegrate it most efficiently [38,50]. The indicator for assessing the effectiveness of network dismantling strategies is described as: the area under the network connectivity degradation curve when nodes are removed one by one in a certain order. Our task objective is to develop the optimal strategy for removing nodes, which entails determining a sequence of nodes to be removed that minimizes the value of Accumulated Normalized Connectivity (ANC) [31,51]:

$$ANC = \frac{1}{N} \sum_{n=1}^N \frac{S(n)}{S} \quad (1)$$

where N denotes the total number of nodes in the network; $S(n)$ represents the network's performance after attacking n nodes and is generally measured by the Largest Connected Component (LCC) size. S represents the initial connectivity of the graph before the removal of any node. The ANC is plotted by taking the horizontal axis as n/N and the vertical axis as $S(n)/S$. The network dismantling problem is fundamentally a combinatorial optimization problem, posing difficulties in finding an optimal solution, particularly in the context of large-scale networks.

Numerous mechanisms and functionalities within the network are significantly influenced by a small number of nodes, commonly recognized as pivotal [38]. Effective dismantling occurs when the removal of a small set of nodes causes the system rapidly decomposes into smaller isolated clusters. Finding the smallest set of nodes driving a system towards collapse has attracted increasing attention for real-world applications.

3. Methodology

This section presents a new GASC model to address the ND problem in detail. Our GASC model mainly consists of the graph attention networks, the shortcut connections mechanism and the multilayer perceptron. Our GASC model can get the high-level features of the nodes by aggregating the features of each node and its neighbors, aiming to get the dismantling score for each node. The higher the dismantling score of a node, the more important it is in the network, and thus the higher the probability that the node should be removed. Then, we design an unsupervised learning approach to train the GASC model, which not only addresses the challenging problem of labeling in supervised learning, but also effectively guides the model to learn the dismantling score for each node. Furthermore, a tailoring method based on the target network is used to generate the training datasets. Our model can effectively learn the underlying features of a network's topological structure and achieve more efficient network dismantling.

3.1. A novel GASC model for the network dismantling problem

We develop a novel GASC model based on the graph attention networks (GAT) [47]. Inspired by the works of [52,53], our model integrates the Shortcut Connections mechanism [48,54] to overcome the vanishing gradients problem of the deep model and improve the network's representation and learning capabilities. The GASC model can effectively learn the underlying features of a network's topological structure and get the dismantling score for each node.

The GASC model, as depicted in Fig. 1, is presented to address the ND problem. The GASC model first adopts the GAT layers and the shortcut connections mechanism to embed node features. Then, our model can obtain the high-level features of the nodes by aggregating the features of each node and its neighbors. Finally, the high-level features are fed into

the Multilayer Perceptron (MLP) [55] to generate the dismantling score vector for the nodes. The details of our GASC model are explained below.

The GASC model first adopts the elementary node features f and the network as the input, and outputs the high-level embedding features matrix $H(f)$ for the nodes. Specifically, it first calculates the elementary node features f (e.g., degree centrality [22], betweenness centrality [25]). Then, the node features f and the network are input into the GAT layers (GAT layer- L , $L=4$), which adopts the multi-head attention mechanism with 4 heads. The results of the multi-head attention mechanism conduct the concatenate and the dropout layers, which is denoted as nonlinear function $F(f)$. Thus, the $H(f)$ is defined as Eq. (2).

$$H(f) = \text{ELU}(F(f, w_i)) \quad (2)$$

where the ELU is activation function; the w_i is trainable parameters; these biases are omitted to simplify the notations in the model.

In traditional neural networks, each layer's output is derived by transforming the previous layer's output using an activation function. Inspired by the works of ResNet [52] and PAM-DenseNet [53], we integrate the shortcut connections mechanism to directly add the output of the previous layer to the output of the subsequent layer. This operation can be understood as introducing an identity mapping into the network. The network is better able to learn the changing parts of the identity mapping without relying entirely on the transformation capability of the network from the beginning. The shortcut connections mechanism can effectively transfer gradients and overcome the vanishing gradients problem, making the network easier to optimize. We use the fully connected layer to adjust the input dimension of the shortcut connections mechanism to match the dimension of $F(f)$. The high-level embedding features matrix $H(f)$ for the nodes can be obtained according to Eq. (3).

$$H(f) = \text{ELU}(F(f, w_i)) + f \bullet w_s \quad (3)$$

where the w_s is trainable parameters.

Then, our GASC model inputs the high-level embedding features matrix $H(f)$ of the nodes, and outputs the dismantling score vector for the nodes. Specifically, the encoded high-level feature information is first passed through the MLP, which comprises the input, hidden and output layers. Then, the information from the output layer is passed through the ELU activation function. Finally, the dismantling score of each node is acquired as shown in Eq. (4). Thus, we can get the dismantling score ranking of the nodes, where a darker color indicates a higher ranking in terms of scores, as illustrated in Fig. 1.

$$S^{\text{dis_score}} = \text{ELU}(H(f) \bullet w_m + b_m) \quad (4)$$

where w_m and b_m are trainable parameters in our model, $S^{\text{dis_score}}$ is dismantling score of the node predicted by our model.

By taking into account the node's L -hop neighborhood, it is easier and more practical to understand how the GASC model, which includes L -layer GAT networks and integrates the shortcut connections mechanism, learns the dismantling score for each node n . We analyze a tree rooted at node n with a height of L , where the neighbors of each node are its child nodes. Specifically, the $(l+1)$ -th level of the tree consists of the neighbor nodes of the l -th level nodes. The high-level node features H_n^l of node n are obtained by aggregating information from the L -hop neighborhood in a bottom-up manner. Our model begins at the bottom of the tree (i.e., from the nodes that are L hops away from node n), computes the high-level node features at layer L for each node, and then moves upwards until it reaches the root node n . This implies that our model is capable of aggregating information from the entire L -hop neighborhood of node n into H_n^L . Due to the self-attention mechanism of the GAT, the model also considers the varying importance of each node within its neighborhood. Our model can learn a weighted sum function from the training data, enabling the effective aggregation of multiple node features. Additionally, the shortcut connections mechanism can be

understood as introducing an identity mapping into the model to overcome the vanishing gradients problem. These high-level features (H_n^L) are then fed into the MLP to get the dismantling score of the node used in our study.

3.2. Designing unsupervised learning to train the GASC model

In order to address the difficult problem of labeling in supervised learning, we design an unsupervised learning method to train the GASC model. This method can effectively guide the model to learn the dismantling score $S^{\text{dis_score}}$ for each node. Inspired by the work of [43], the goal of the network dismantling problem is to identify and remove the smallest set of nodes while leading to the most subnetworks disconnected, each of which is as small as possible. Considering this, our optimization objective (*Loss*) aims to minimize both the expected number of unaffected nodes after removing the attacked set of nodes (*Loss1*) and the expected number of attacked set of nodes (*Loss2*). Thus, the *Loss* function of the trained GASC model is defined as Eq. (5).

The cascade effects indicate that removing a node in the network can not only affect the connectivity of its local structure, but also trigger further cascaded failures through its neighbor nodes. We remove the smallest possible set of nodes to maximize the cascading impact on as many neighbors as possible. Therefore, the *Loss1* is defined as Eq. (6). Then, the expectation of the minimized target dismantling set (*Loss2*) is calculated by using the summation of dismantling scores acting as a regulatory factor, which is defined as Eq. (7), enforcing our model to output a reasonable solution.

$$\text{Loss} = \text{Loss1} + \text{Loss2} \quad (5)$$

$$\text{Loss1} = \sum_{x \in V} \prod_{y \in \text{Nei}(x)} \left(1 / \left(1 + S_y^{\text{dis_score}} \right) \right) \quad (6)$$

$$\text{Loss2} = \sum_{x \in V} S_x^{\text{dis_score}} \quad (7)$$

where $\text{Nei}(x)$ represents the set of one-hop neighbors of node x . $S_y^{\text{dis_score}}$, $S_x^{\text{dis_score}} \in R^N$ is the dismantling score for each node predicted by the GASC model, which takes into account the potential influence of the node's neighbors on the node. Note that we utilize $\prod_{y \in \text{Nei}(x)} \left(1 / \left(1 + S_y^{\text{dis_score}} \right) \right)$ to measure the collective effect of unaffected nodes in the network when removes the node x . When a node is removed, the collective effect on unaffected nodes is smaller, indicating a larger collective effect on affected nodes.

The results are obtained by calculating the backward-propagated gradients of the *Loss* function according to Eqs. (2), (4), (5), (6), and (7) as shown in Eq. (8), and according to Eqs. (3), (4), (5), (6), and (7) as shown in Eq. (9). As indicated in Eq. (8), when the gradient is backward-propagated from the deep network to the shallow network, the unfortunate consequence is the exponential decay of the obtained gradient update information, commonly known as the vanishing gradients problem. As shown in Eq. (9), when the shortcut connections mechanism is integrated into the model, the output from the shallow network is fed into the deep network through identity mapping. The gradient of the deep network can be directly transferred to the shallow network, effectively preventing the vanishing gradients problem. The network transforms some multiplicative operations into additions during the gradient update, enhancing computational efficiency.

$$\frac{\partial \text{Loss}}{\partial f} = \frac{\partial \text{Loss}}{\partial S^{\text{dis_score}}} \bullet \frac{\partial S^{\text{dis_score}}}{\partial H} \bullet \frac{\partial F}{\partial f} \quad (8)$$

$$\frac{\partial \text{Loss}}{\partial f} = \frac{\partial \text{Loss}}{\partial S^{\text{dis_score}}} \bullet \frac{\partial S^{\text{dis_score}}}{\partial H} \bullet \left(\frac{\partial F}{\partial f} + w_s \right) \quad (9)$$

Table 1

Statistics characteristics of these real-world networks.

Transportation networks	Nodes	Edges	Maximum degree	Average degree
Global liner shipping network [2]	907	14,347	293	31.64
Anaheim traffic [61]	416	634	7	3.05
Berlin_Friedrichshain_roads [62]	224	376	8	3.36
Berlin_Prenzlauerberg_Center_roads [62]	352	554	6	3.15
Eastern Massachusetts traffic [63]	74	129	12	3.49
Contiguous USA [64]	49	107	8	4.37
US airports [64]	1574	17,215	314	21.87
OpenFlights (Opsahl) [64]	2939	15,677	242	10.67

3.3. Tailor the target network to generate the training networks

The capabilities of the model are primarily acquired through learning from the datasets for machine learning, where the information is encoded into the model's parameters [56]. Therefore, the choice of training dataset greatly influences the performance of the model. Most methods of network dismantling rarely consider selecting the training networks with similar domains or characteristics to the target network [38]. This paper is inspired by the work of [31], which adopts the configuration model to generate the training dataset that exhibit similar degree distributions to the target network. Our model incorporates a network tailoring method, which maintains the average degree and degree distributions of the target network. This network tailoring method can generate small synthetic training networks corresponding to the target network.

The network tailoring method is described as follows:

Step 1: Calculate the average degree, denoted as A_d , and the degree distribution, denoted as $P(k)$, of the target network. $P(k)$ refers to the probability of nodes having a degree of k . The node's degree is normalized to a range of $[0,1]$ relative to the maximum degree in the network.

Step 2: Sample each degree multiple times ($m_t = 50$) from the degree distribution $P(k)$ of the target network by using the roulette wheel selection method [57]. It's important to note that the sampled degree is a normalized value, which is multiplied by the expected maximum degree in the sampled degree sequence to ensure that the sampled degree sequence has a similar average degree A_d to the original degree sequence.

Step 3: Determine whether the sum of the sampled degree sequence is even. If the conditions are not met, then return to step 2. If the conditions are met, then the degree sequence closest to average degree of the target network is selected from multiple sampled degree sequences as the input of the Configuration Model (CM) [49]. The CM outputs the small-scale synthetic network with a range of 30 to 50 nodes that is expected to have a similar degree distribution and average degree to the target network.

4. Experiments and results

We selected synthetic networks and 8 real-world networks to test our approach's performance. The effectiveness of the approach is evaluated using the ANC from Eq. (1) in this paper. A smaller ANC value indicates greater effectiveness. Additionally, we evaluate the efficiency of the approach by considering its runtime. Table 1 presents the characteristics and statistics of these real-world networks, including the nodes, edges, maximum degree, and average degree. These real-world networks examined in our study display a diverse array of topologies and span a range of sizes. These networks have been processed to remove self-loop edges and isolated nodes. Additionally, we utilize two classic networks namely the degree-homogeneous small-world network (Watts-Strogatz, WS) [58] and the degree-heterogeneous scale-free (Barabási-Albert, BA) network [59] to analyze the influence of network structure on the proposed approach. Meanwhile, we also consider analyzing the influence of complex networks with community structure [60] on the performance of

Table 2

The GASC model configurations and hyper-parameters.

Hyper-parameter	Value
Maximum training epochs	300
Probability of dropout	0.2
Adam optimizer learning rate	10^{-4}
Learning decay	10^{-5}
Batch size	128
Negative slope of Leaky ReLU	0.2
Number of self-attention heads	4
Number of GAT and MLP layers	4
Number of neurons per layer	64, 32, 16, 8
Dimensions of node embedding	64, 64, 64, 64

our method.

The hyperparameters' values in our GASC model have a significant impact on the model's performance. Therefore, it is necessary to select appropriate parameters' values, including the maximum training epochs, the probability of dropout, the Adam optimizer learning rate, the learning decay, the batch size, the negative slope of Leaky ReLU, the number of self-attention heads, the number of GAT and MLP layers, the number of neurons per layer, and the dimensions of node embedding. The experiments are performed using a computer equipped with an Intel i7-12700 CPU and 2.10 GHz. The optimal GASC model configurations and hyperparameters are shown in Table 2. Specifically, the parameter for maximum training epochs is determined based on a comprehensive consideration of the model's convergence characteristics, dataset size, and the need to balance time cost. In practice, if the model fails to achieve satisfactory performance within the maximum epochs, the number of epochs can be increased appropriately. Conversely, if the model converges earlier, the maximum epochs can be reduced to improve efficiency. On some datasets, we find that increasing the batch size can continue to improve the performance of the method. However, due to memory limitations, we set the batch size to 128. Higher-dimensional node embeddings and more self-attention heads can capture richer feature information but will increase computational complexity. Similarly, more neurons per layer and deeper network layers can learn more complex features, thereby enhancing the model's expressive power, but they also come with higher computational costs and an increased risk of overfitting. The parameter for the negative slope of Leaky ReLU is set in the range of (0.1, 0.5). Increasing it further would weaken the non-linear capability of the activation function. The dropout probability, the Adam optimizer learning rate and the learning decay are key optimization parameters of the model. We analyze the sensitivity of the GASC model's performance to hyperparameter tuning. The evaluation of our model's performance involves observing and analyzing the decline curves of the training loss function during the training procedure, as shown in Fig. A.1 of Appendix A. The horizontal axis represents the number of training epochs, with a maximum of 300 epochs, while the vertical axis represents the corresponding training loss values for each epoch. We conducted multiple experiments on the key hyperparameter of the model to search for the optimal parameter values. From Fig. A.1 in Appendix A, it can be clearly observed that the optimal parameter values for the dropout probability, the learning decay, and

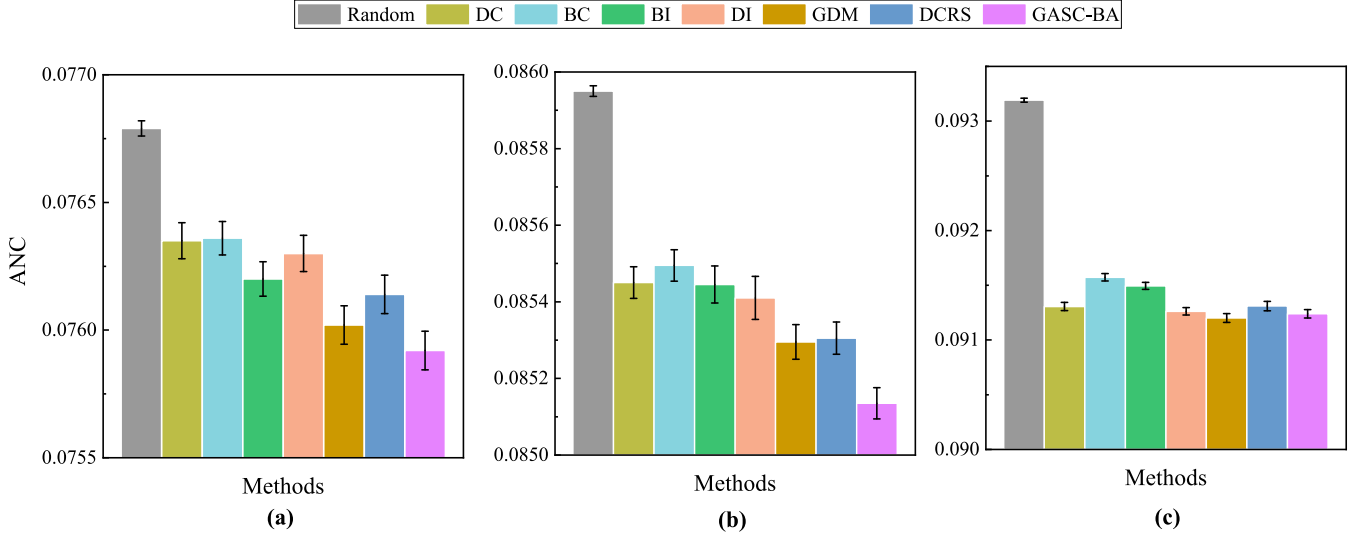


Fig. 2. Effectiveness comparison of eight methods on synthetic networks. (a) BA networks with 50 nodes. (b) BA networks with 100 nodes. (c) BA networks with 500 nodes.

the Adam optimizer learning rate in our model are 0.2, 10^{-5} , and 10^{-4} , respectively (indicated by the red line).

4.1. Effectiveness comparison

Removing the smallest set of nodes in the real-world system may cause a very large impact on the stability of the entire system. Most real-world networks analyzed in this study exhibit a certain degree of heterogeneity, which is similar to the BA model. Our GASC model utilizes the BA model during training, referred to as GASC-BA. We use the BA model to generate 10,000 training networks, with each small network's node size randomly chosen from a range of 30 to 50. The time required for this process is 10.87 s. The GASC-BA method can effectively identify a few nodes sequences in the network to dismantle networks. To further improve the performance of the GASC-BA, we use the method of tailoring the target network to generate the training networks, referred to as GASC-CM. Unsupervised learning is adopted to train our GASC model. We use the target network tailoring method to generate 10,000 training networks, which the time required is shown in Table B.1 of the Appendix B.

The GASC-BA and GASC-CM are compared against the random node attack and various state-of-the-art baselines: (1) Degree Centrality (DC) [22], which attacks nodes as degree decreases. The degree centrality serves as the most straightforward metric for assessing node centrality within network analysis. The greater node degree within a network corresponds to higher degree centrality, signaling heightened significance of the node within the network. (2) Betweenness Centrality (BC) [25], which is a metric of a node's importance by quantifying the number of shortest paths that traverse through it. (3) Interactive Betweenness (BI) [19], which attacks nodes with the highest betweenness centrality but recalculates betweenness centrality in each iteration. The BI is currently the indicator for judging the effectiveness of most ND algorithms [11,19]. (4) Interactive Degree (DI), which attacks nodes with the highest degree but recalculates degree in each iteration. (5) GDM¹ [38], which is an effective deep-learning method based on supervised learning. (6) DCRS² [43], which is a competitive deep-learning method based on unsupervised learning.

We analyze the effectiveness of various methods on both synthetic

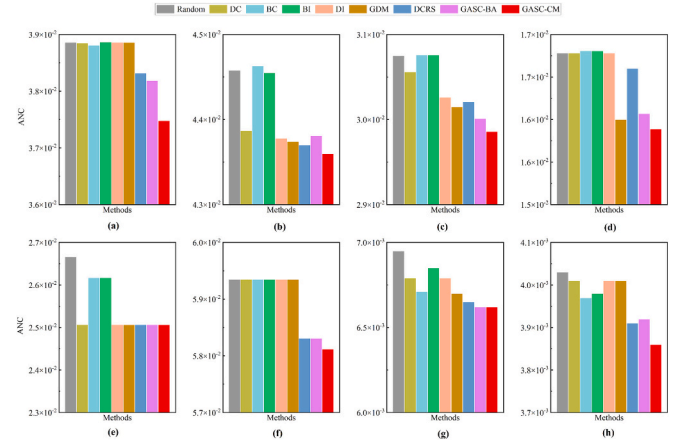


Fig. 3. Effectiveness comparison of nine methods on 8 real-world networks. (a) Global liner shipping network. (b) Anaheim traffic. (c) Berlin Friedrichshain_roads. (d) Berlin_Prenzlauerberg_Center_roads. (e) Eastern Massachusetts traffic. (f) Contiguous USA. (g) US airports. (h) OpenFlights (Opsahl).

networks and real-world networks. Firstly, Fig. 2 shows the ANC on synthetic BA networks with 50, 100, and 500 nodes according to Eq. (1). For each scale, we randomly generate 10 instances and report their average results under attacking 10 % of the nodes. The error bars in Fig. 2 represent the standard deviation of 10 random instances. Note that the GASC-BA is trained on the BA networks with node sizes ranging from 30 to 50. From Fig. 2, it can be found that the GASC-BA outperforms other approaches for synthetic BA networks with 50 and 100 nodes. The performance of GASC-BA is slightly inferior to the GDM on synthetic BA networks with 500 nodes. The GDM method is supervised learning, and the synthetic BA networks we evaluate, as well as the training network, are from the same distribution. Thus, the GDM method may produce superior results. The GASC-BA outperforms the Random, DC, BC, BI, DI and unsupervised learning DCRS in solving the synthetic network problems of various scales.

Then, Fig. 3 shows the ANC on 8 real-world networks using nine methods according to Eq. (1). It can be intuitively observed from Fig. 3 that for 8 real-world networks, the GASC-BA achieves better performance compared to methods Random, DC, BC, BI, DI, GDM, and DCRS on 3 networks. The DC, DI, GDM, DCRS and GASC-BA achieve the same

¹ <https://github.com/NetworkScienceLab/GDM>

² <https://github.com/JiazhenZhang/DCRS>

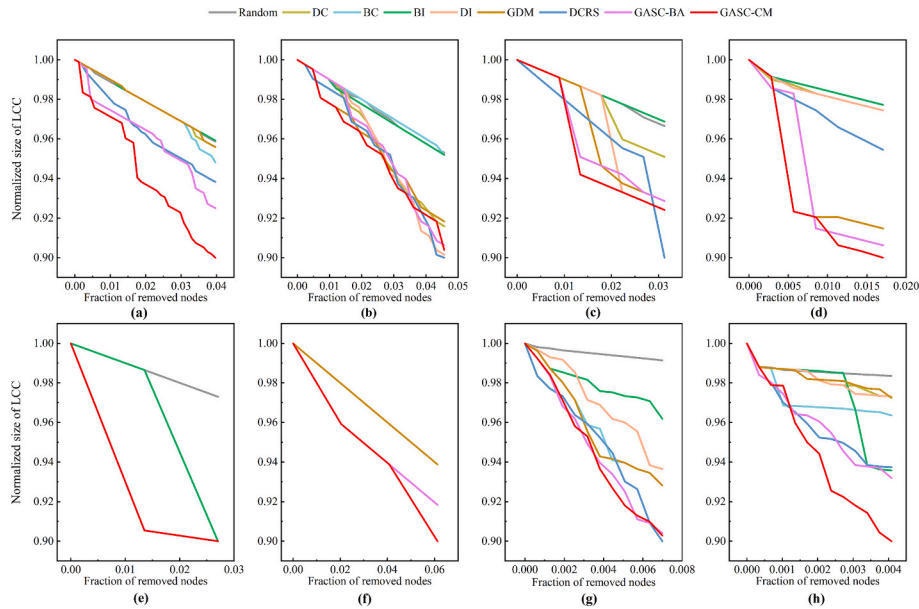


Fig. 4. The normalized size of LCC under different attack strategies for 8 real-world networks. (a) Global liner shipping network. (b) Anaheim traffic. (c) Berlin - Friedrichshain_roads. (d) Berlin_Prenzlauerberg_Center_roads. (e) Eastern Massachusetts traffic. (f) Contiguous USA. (g) US airports. (h) OpenFlights (Opsahl).

Table 3
Effectiveness comparison of nine methods in network disturbance scenarios.

Networks		Global liner shipping network			Anaheim traffic		
		0.1%	0.5%	1%	0.1%	0.5%	1%
ANC Methods	Random	0.05665	0.05665	0.05665	0.04458	0.04458	0.04436
	DC	0.05656	0.05664	0.05670	0.04387	0.04379	0.04401
	BC	0.05610	0.05616	0.05640	0.04463	0.04458	0.04440
	BI	0.05666	0.05665	0.05668	0.04455	0.04463	0.04442
	DI	0.05656	0.05664	0.05670	0.04378	0.04370	0.04373
	GDM	0.05652	0.05660	0.05667	0.04381	0.04359	0.04381
	DCRS	0.05624	0.05630	0.05640	0.04372	0.04356	0.04351
	GASC-BA	0.05564	0.05624	0.05638	0.04392	0.04365	0.04355
	GASC-CM	0.05556	0.05589	0.05570	0.04372	0.04351	0.04341

Note: the results in bold indicate the best effectiveness achieved by all methods.

performance on the Eastern Massachusetts traffic. Similarly, the DCRS and GASC-BA achieve the same performance on the Contiguous USA. The performance of GASC-BA is slightly inferior to the DCRS on the Anaheim traffic and OpenFlights (Opsahl). The performance of GASC-BA is slightly inferior to the GDM on the Anaheim traffic and Berlin_Prenzlauerberg_Center_roads. The GASC-CM further improves the performance of GASC-BA, except that the two methods achieve the same performance on the Eastern Massachusetts traffic and US airports. In summary, the GASC-CM outperforms other methods in network dismantling, while the GASC-BA ranks second in dismantling performance.

In order to more intuitively characterize the impact on network dismantling when using different methods to attack nodes, Fig. 4 illustrates the normalized size of Largest Connected Component (LCC) under different attack strategies for 8 real-world networks. The horizontal axis represents the fraction of nodes removed relative to the total nodes. The vertical axis represents the size of LCC after node removal, divided by the initial size of LCC before node removal. It can be observed from

Fig. 4 that the GASC-CM exhibits the earliest and fastest degradation in network performance compared to other methods upon removal of the attacked node. This observation indicates that the GASC-CM can effectively identify the few nodes removal order in the network and dismantle the network.

Networks are dynamically changing in the real world. We simulate network disturbance scenarios by randomly adding and removing a certain proportion of edges or nodes from networks such as the Global liner shipping network and Anaheim traffic. The network's fundamental structure remains approximately consistent in network disturbance scenarios. Therefore, the GASC-CM does not need to re-tailor the target network to generate the training network. The GASC-BA and GASC-CM both utilize pre-trained models to solve the disturbed Global liner shipping network and Anaheim traffic. The number of changing edges is 0.1 %, 0.5 % and 1 % of the total number of edges, which is 15, 72 and 144 respectively in Global liner shipping network, and is 1, 4 and 8 respectively in Anaheim traffic. Table 3 presents the effectiveness comparison of nine methods in network disturbance scenarios. The

Table 4
Efficiency comparison of six methods on real-world networks.

Time Methods	BI	DI	GDM	DCRS	GASC-BA	GASC-CM
Global liner shipping network	3m27.781s	3.609s	0.050s	0.247s	0.051s	0.055s
Anaheim traffic	6.106s	0.850s	0.007s	0.029s	0.007s	0.013s
Berlin_Friedrichshain_roads	0.865s	0.255s	0.009s	0.141s	0.005s	0.009s
Berlin_Prenzlauerberg_Center_roads	3.267s	0.611s	0.008s	0.088s	0.009s	0.010s
Eastern Massachusetts traffic	0.044s	0.030s	0.005s	0.029s	0.005s	0.008s
Contiguous USA	0.023s	0.013s	0.004s	0.026s	0.004s	0.004s
US airports	9m31.279s	13.834s	0.064s	0.475s	0.106s	0.108s
OpenFlights (Opsahl)	1h13m30.681s	47.077s	0.105s	0.942s	0.205s	0.225s

Note: the results in bold indicate the best efficiency achieved by all methods.

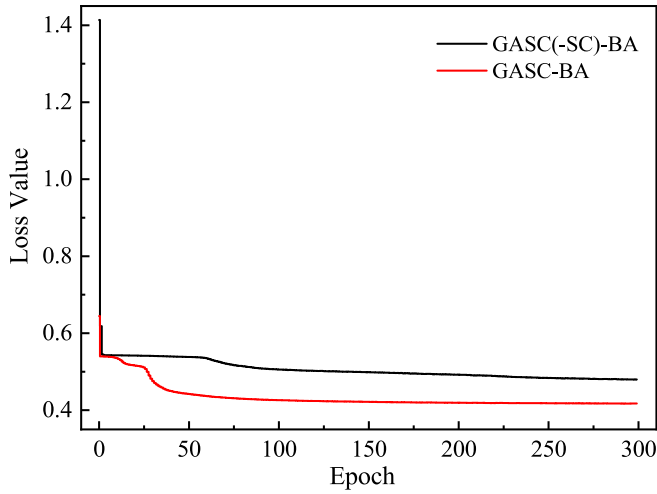


Fig. 5. Visualizations depict the decline curves of the training loss function.

results in bold indicate the best performance achieved by all methods. The results in Table 3 indicate that GASC-CM achieved the best performance across all networks. GASC-BA ranks second in the disturbed Global liner shipping network, while DCRS ranks second in the disturbed Anaheim traffic.

To vividly depict the attack process of each method, Fig. C.2 in the Appendix C shows the normalized size of LCC under the node attack using various methods for the disturbed Global liner shipping network and Anaheim traffic. It can be found from Fig. C.2 that the performance of GASC-CM is better than GASC-BA and other methods, indicating that our approach is effective and robust for disturbed networks. In summary, our approach achieves outstanding performance in network disturbance scenarios, demonstrating its strong robustness.

4.2. Efficiency comparison

We then analyze the efficiency of node attack strategies using different methods to dismantle networks. The BI and DI methods offer relatively good quality for attacking nodes. Deep learning-based methods are tested during runtime using pre-trained models. Table 4 compares the efficiency of six dismantling methods across 8 real-world networks. The results in bold indicate the highest efficiency achieved among all methods.

From Table 4, it can be observed that the methods based deep learning, namely GDM, DCRS, GASC-BA and GASC-CM, have high solving efficiency, which are greatly superior to the BI and DI methods.

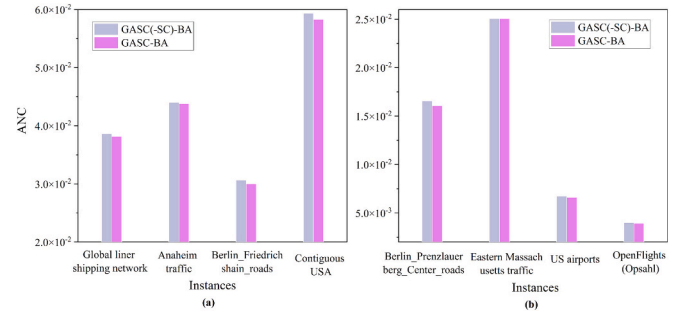


Fig. 6. Performance comparison of GASC(SC)-BA and GASC-BA for real-world networks. (a) Global liner shipping network, Anaheim traffic, Berlin_Friedrichshain_roads, and Contiguous USA. (b) Berlin_Prenzlauerberg_Center_roads, Eastern Massachusetts traffic, US airports and OpenFlights (Opsahl).

Although the solution efficiency of the GASC-BA is slightly worse than the GDM, it is better than the GASC-CM and DCRS. Therefore, for the problem of network dismantling, the GASC-BA should be selected when the solution efficiency is required, while the GASC-CM should be selected when the solution effectiveness is required.

4.3. Model performance validation with the shortcut connections mechanism

Our GASC model and the model without shortcut connections mechanism both utilize the BA model during training, and they are respectively referred to as GASC-BA and GASC(SC)-BA. Both GASC-BA and GASC(SC)-BA are trained using unsupervised learning.

The evaluation of our model's performance involves observing and analyzing the decline curves of the training loss function during the training procedure, as depicted in Fig. 5. The horizontal axis represents the number of training epochs, with a maximum of 300 epochs, while the vertical axis represents the corresponding training loss values for each epoch. From Fig. 5, it can be clearly observed that the GASC-BA achieves faster convergence compared to the GASC(SC)-BA. The advantage of rapid convergence lies in achieving a stable performance level with fewer training iterations, thereby optimizing the utilization of time and computational resources. More importantly, the GASC-BA exhibits considerably lower training error compared to the GASC(SC)-BA, which indicates that the vanishing gradients problem is well addressed in this scheme.

Furthermore, to verify the GASC model's generalization performance on the test set, we conducted further experimental validation. We use the GASC-BA and GASC(SC)-BA to calculate the ANC of 8 real-world networks according to Eq. (1), and the results are shown in Fig. 6.

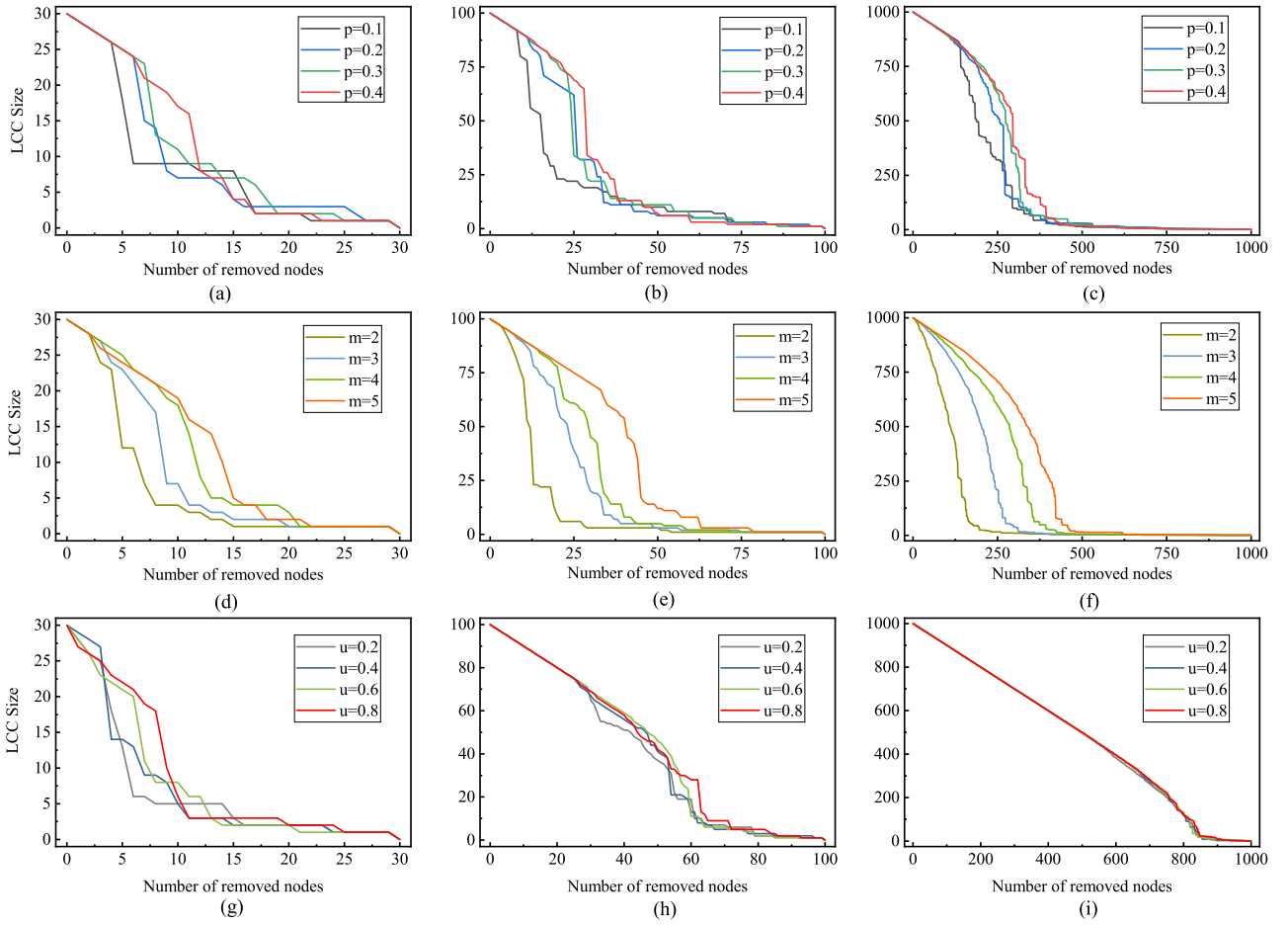


Fig. 7. The influence of network structure on dismantling performance under sequential attacks for small-scale and medium-scale WS networks ((a), (b) and (c)), small-scale and medium-scale BA networks ((d), (e) and (f)), and small-scale and medium-scale LFR networks ((g), (h), and (i)) with 30, 100 and 1000 nodes.

Attacking just a small fraction of nodes in real-world systems can cause severe disruption in the entire system.

As shown in Fig. 6, both GASC(SC)-BA and GASC-BA achieve the same dismantling performance for the Eastern Massachusetts traffic network among 8 real-world networks. However, the GASC-BA outperforms the GASC(SC)-BA for the other 7 networks. Incorporating the shortcut connections mechanism can overcome the vanishing gradients problem in our model, thereby improving its representation and learning capabilities. This improvement enables accurate predictions even on previously unseen networks. Considering the analysis of the descent curves of the training loss function and the generalization performance on real-world networks, we can deduce that the GASC-BA exhibits superior performance to the GASC(SC)-BA.

4.4. Influence of network structure in complex networks

The network structure, serving as the backbone of all infrastructure, holds significant importance in analyzing network vulnerabilities. Two of the typical complex networks, the WS network and the BA network, have been extensively investigated in prior research. We examine the influence of topological parameters in network structures, specifically the probability of rewiring each edge (p) in WS networks and the number of edges to attach from a new node to existing nodes (m) in BA networks, on our approach. We employ the Watts-Strogatz model [58] to generate the WS network, with p values set to 0.1, 0.2, 0.3, and 0.4. Similarly, we use the Barabási-Albert model [59] to generate the BA network, with m values set to 2, 3, 4, and 5. The Lancichinetti-Fortunato-Radicchi (LFR) model [65] is used to generate synthetic complex

networks with community structure, referred to as LFR networks. Then, we perform our method on LFR networks with different levels of community structure (i.e., fraction of inter-community edges incident to each node, represented by the parameter u in the LFR model). When generating LFR networks, we set the value of the parameter u to 0.2, 0.4, 0.6, and 0.8.

Then, we train a GASC-BA model on small synthetic networks, with each small network's node size randomly chosen from a range of 30 to 50. The trained model is then generalized to solve small-scale and medium-scale WS, BA, and LFR networks with 30, 100 and 1000 nodes, as well as large-scale and very-large-scale WS, BA, and LFR networks with 5000, 10,000, and 15,000 nodes. Thus, the computational complexity of our method in solving very-large-scale networks is dominated by calculating the initial features of the very-large-scale network (i.e., degree centrality and betweenness centrality in our study). Fig. 7 depicts the influence of topological parameters p , m and u on the dismantling performance of small-scale and medium-scale networks with 30, 100 and 1000 nodes under sequential attacks. Fig. 8 depicts the influence of topological parameters p , m and u on the dismantling performance of large-scale and very-large-scale networks with 5000, 10,000 and 15,000 nodes under sequential attacks. The results depicted in Fig. 7 and Fig. 8 indicate that as the parameters p , m and u increase, the performance of our method gradually deteriorates. It indicates that the effectiveness of sequence attacks in complex networks is impacted by topological parameters p , m and u . This phenomenon can be explained through the lens of network connectivity.

In the WS network, attacked nodes will directly impact their neighboring nodes due to local connections. Hence, considering the cascade

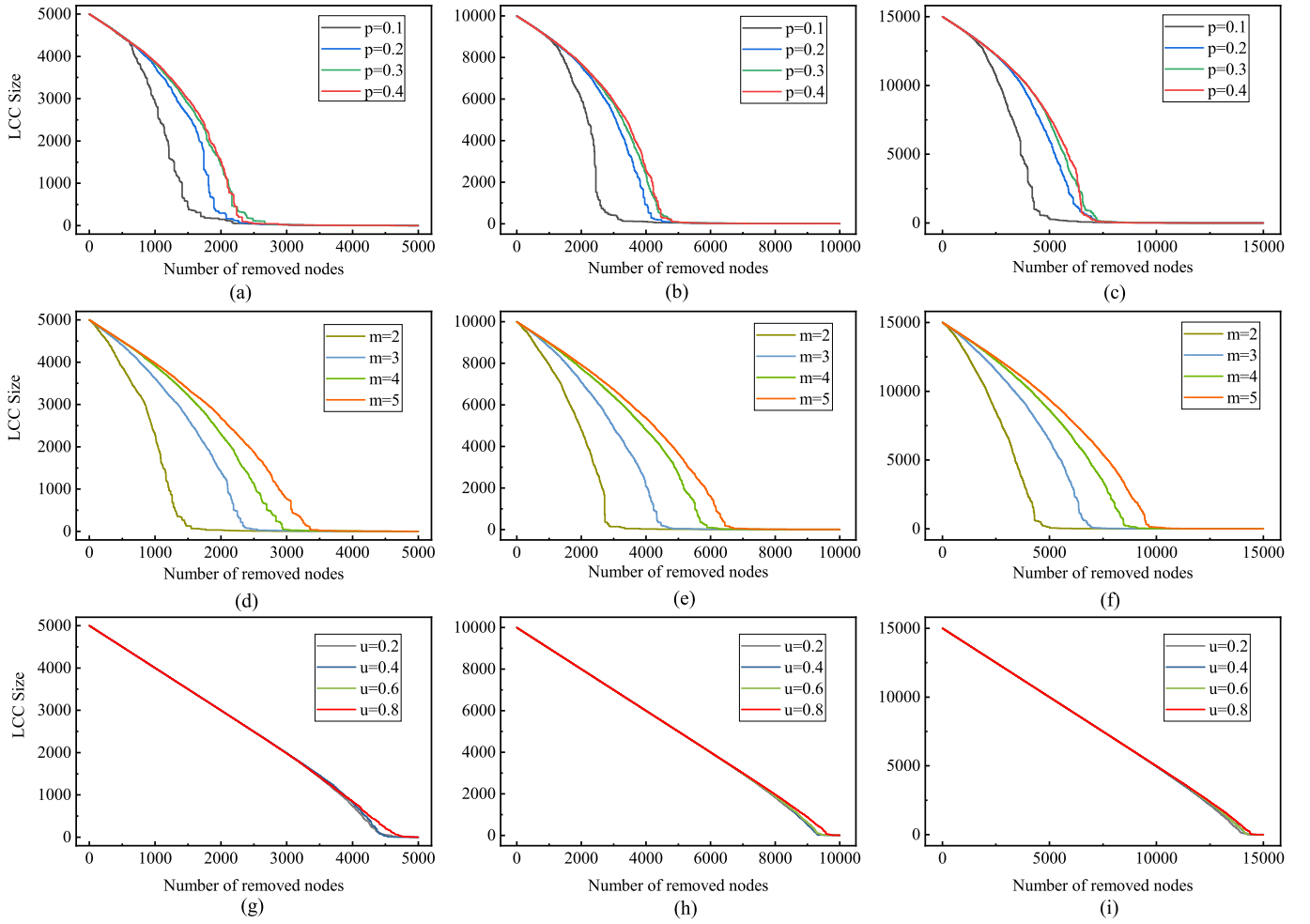


Fig. 8. The influence of network structure on dismantling performance under sequential attacks for large-scale and very-large-scale WS networks ((a), (b) and (c)), large-scale and very-large-scale BA networks ((d), (e) and (f)), and large-scale and very-large-scale LFR networks ((g), (h), and (i)) with 5000, 10,000 and 15,000 nodes.

effects of nodes, the loss function we designed can effectively guide the GASC model to learn the optimal set of node removal for dismantling the network. However, as the topological variable p increases, the WS network gradually evolves into the random network, where the node features and connectivity are similar. This may lead to a more uniform cascade effect within the network. As a result, the difference in disintegration scores between nodes learned by the GASC model becomes less significant as guided by our loss function. Consequently, the algorithm's performance degrades under the same training process. In the BA network, the network connectivity is highly uneven, with the majority of nodes having few neighbors, while a small number of nodes located in hub positions. As the topological parameter m increases, the network's heterogeneity decreases, suggesting that the majority of nodes tend to have more moderate connectivity. Therefore, this explanation is similar to that of the WS network. In the LFR network, as the parameter u increases, the number of inter-community connections increases and the boundaries between communities gradually become blurred. That is, the community structure of the network weakens, making the overall network more homogeneous. Consequently, removing only a few key nodes may not significantly reduce the network's connectivity, thereby making network dismantling increasingly challenging. In summary, as the network topological parameters p , m and u increase, the dismantling performance of our approach gradually decreases. This indicates that our method performs better as the heterogeneity of these three types of networks becomes more pronounced.

Moreover, the results depicted in Figs. 7 and 8 indicate that our

model achieves outstanding performance in solving the WS and BA networks, but the disintegration performance of our method progressively deteriorates when dealing with large-scale and very-large-scale LFR networks. Although our method is less suitable for disintegration of large-scale and very-large-scale LFR networks, it demonstrates effectiveness in addressing disintegration of small-scale and medium-scale LFR networks. Additionally, note that the Global liner shipping network selected in our study is a real-world maritime transportation network with significant community structure characteristics [2]. The application of our method to the Global liner shipping network achieves excellent performance, as shown in Figs. 3(a) and 4(a).

5. Conclusions

Understanding and analyzing the structural vulnerability of real-world networks is a significant challenge in modern society. This issue can be addressed by tackling the network dismantling problem. This paper presents a new GASC model utilizing unsupervised learning to learn the complex topological structure for dismantling networks. The GASC model primarily consists of the graph attention network, the shortcut connections mechanism and the multilayer perceptron. Our GASC model can obtain the high-level features of the nodes by aggregating the features of each node and its neighbors. The shortcut connections mechanism is integrated into our model to overcome the vanishing gradients problem, aiming to improve the network's representation and learning capabilities. We design an unsupervised learning

method to train the GASC model, which can effectively guide the model to learn the dismantling score for each node in the network. The experiments validate the effectiveness and efficiency of our approach across synthetic networks and various real-world networks by adopting the BA model to generate generally small synthetic training networks (named GASC-BA).

Furthermore, to further improve the performance of our approach, we use a target network tailoring method to generate the training networks that preserves important characteristics of the target network (named GASC-CM). We performed comprehensive experiments on real-world networks. The results validate that our approach is significantly superior to current state-of-the-art approaches and demonstrates strong robustness.

Our network dismantling method GASC can effectively assess the structural vulnerability of real-world networks in the face of unforeseen disasters or accidents, thus providing valuable decision-making insights for policymakers and infrastructure planners. Therefore, our approach has broad potential applications in supporting the stable operation of critical real-world networked systems such as transportation, power grids, logistics and communications.

The proposed approach is currently applicable to solving the dismantling problem in static networks but cannot yet be directly generalized to temporal or multi-layer networks. A common simplification approach is to represent temporal networks as snapshot networks, which can be expressed as a series of static graphs. For example, Yu et al. [66] proposed a novel MLI method that combines network embedding and machine learning to identify critical nodes in temporal networks. This method abstracts temporal networks into snapshot networks and represents temporal networks using a multilayer network structure. In this method, one crucial aspect is properly generating feature matrices for nodes—constructing a feature matrix for each node based on its neighborhoods in all snapshots as input feature matrices for the training model. Inspired by the work of Yu et al. [66], to generalize our method to the dismantling of temporal and multi-layer networks, the key technical challenge lies in node embedding, specifically how to construct a feature matrix for each node based on its neighborhoods in all snapshots. Furthermore, considering the inherent similarity among snapshot

networks across different time windows in a temporal network, our future work is expected to leverage data from the first x time windows as a training dataset to predict the network dismantling outcomes for the subsequent time windows.

CRediT authorship contribution statement

Li Hong: Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Mengqiao Xu:** Writing – review & editing, Writing – original draft, Validation, Supervision, Resources, Project administration, Methodology, Investigation, Funding acquisition, Formal analysis, Data curation, Conceptualization. **Yu Liu:** Writing – review & editing, Supervision, Resources, Investigation, Funding acquisition, Formal analysis. **Xuemei Zhang:** Visualization, Validation, Methodology, Investigation, Formal analysis, Data curation. **Changjun Fan:** Writing – review & editing, Visualization, Resources, Methodology, Funding acquisition, Formal analysis.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

M.X. acknowledges the support from the National Natural Science Foundation of China (Grant: 72101046). Y.L. acknowledges the support from the National Natural Science Foundation of China (Grant: 61672128) and from the Dalian Key Field Innovation Team Support Plan (Grant: 2020RT07). C.F. acknowledges the support from the National Natural Science Foundation of China (Grant: 62206303), from the Science and Technology Innovation Program of Hunan Province (Grant: 2023RC3009), and from the Foundation Fund Program of National University of Defense Technology (Grant: JS24-05).

Appendix A

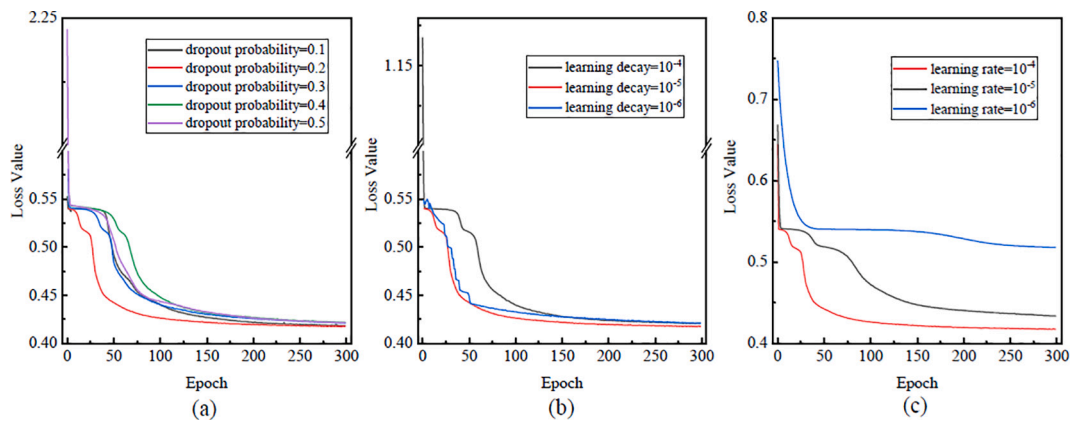


Fig. A.1. The sensitivity of the GASC model's performance to hyperparameter tuning. (a) the dropout probability. (b) the learning decay. (c) the Adam optimizer learning rate.

Appendix B

Table B.1
Time required to generate 10,000 training networks using the target network tailoring method.

Networks	The time of network tailoring (m)
Global liner shipping network	49.11
Anaheim traffic	47.45
Berlin_Friedrichshain_roads	47.19
Berlin_Prenzlauerberg_Center_roads	47.33
Eastern Massachusetts traffic	47.14
Contiguous USA	47.10
US airports	49.52
OpenFlights (Opsahl)	50.14

Appendix C

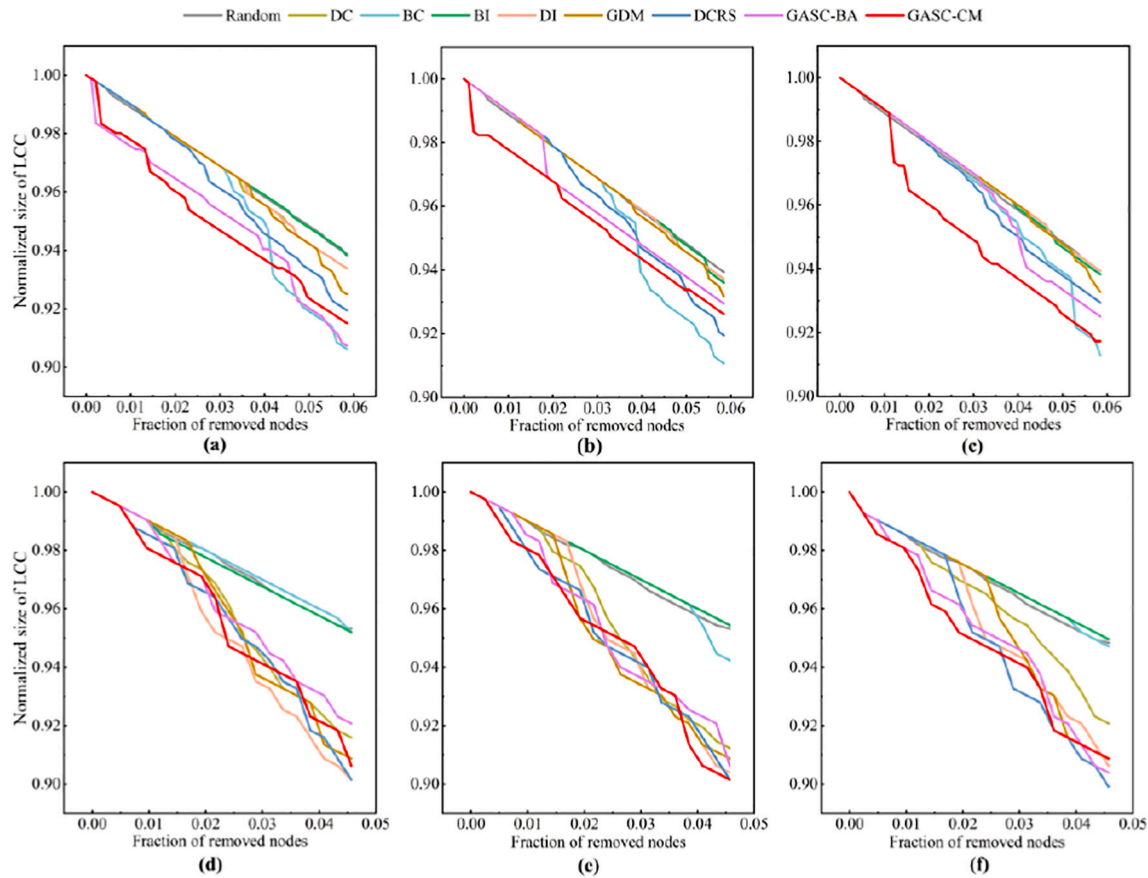


Fig. C.2. The normalized size of LCC under different attack strategies for disturbed Global liner shipping network and Anaheim traffic. (a) Global liner shipping network changes 0.1 % of the edges. (b) Global liner shipping network changes 0.5 % of the edges. (c) Global liner shipping network changes 1 % of the edges. (d) Anaheim traffic changes 0.1 % of the edges. (e) Anaheim traffic changes 0.5 % of the edges. (f) Anaheim traffic changes 1 % of the edges.

Data availability

The code and data will be made available on request.

References

[1] Hassan SA, Amlan HA, Alias NE, Ab-Kadir MA, Sukor NSA. Vulnerability of road transportation networks under natural hazards: a bibliometric analysis and review. *Int J Disaster Risk Reduct* 2022;83:103393. <https://doi.org/10.1016/j.ijdrr.2022.103393>.

[2] Xu M, Pan Q, Muscoloni A, Xia H, Cannistraci CV. Modular gateway-ness connectivity and structural core organization in maritime network science. *Nat Commun* Dec. 2020;11(1):2849. <https://doi.org/10.1038/s41467-020-16619-5>.

[3] Tsiotas D, Polyzos S. Decomposing multilayer transportation networks using complex network analysis: a case study for the Greek aviation network. *J Complex Netw* Dec. 2015;3(4):642–70. <https://doi.org/10.1093/comnet/cnv003>.

[4] Quinn A, Woehle R, Tiemann K. Social network analysis for analyzing groups as complex systems. *J Soc Serv Res* Oct. 2012;38(5):605–18. <https://doi.org/10.1080/01488376.2012.721278>.

[5] Wang S, Guo Z, Huang X, Zhang J. A three-stage model of quantifying and analyzing power network resilience based on network theory. *Reliab Eng Syst Saf* 2024;241(September 2023):109681. <https://doi.org/10.1016/j.res.2023.109681>.

- [6] Cohen R, Erez K, Ben-Avraham D, Havlin S. Resilience of the internet to random breakdowns. *Phys Rev Lett* 2000;85(21):4626–8. <https://doi.org/10.1103/PhysRevLett.85.4626>.
- [7] Albert R, Jeong H, Barabási AL. Error and attack tolerance of complex networks. *Nature* 2000;406(6794):378–82. <https://doi.org/10.1038/35019019>.
- [8] Luke TW. Power loss or blackout: the electricity network collapse of august 2003 in North America. *Disrupted cities When Infrastruct Fail* 2010:55–68.
- [9] Yang S, Hou Z, Chen H. Evaluation of vulnerability of MAV/UAV collaborative combat network based on complex network. *Chaos Solit Fract* 2023;172, no. May: 113500. <https://doi.org/10.1016/j.chaos.2023.113500>.
- [10] Xu M, Deng W, Zhu Y, Lü L. Assessing and improving the structural robustness of global liner shipping system: a motif-based network science approach. *Reliab Eng Syst Saf* 2023;240:109576. <https://doi.org/10.1016/j.res.2023.109576>.
- [11] Wandelt S, Shi X, Sun X. Estimation and improvement of transportation network robustness by exploiting communities. *Reliab Eng Syst Saf* Feb. 2021;206:107307. <https://doi.org/10.1016/j.res.2020.107307>.
- [12] Cicchini T, Caridi I, Ermann L. Robustness of the public transport network against attacks on its routes. *Chaos Solit Fract* 2024;184, no. May:115019. <https://doi.org/10.1016/j.chaos.2024.115019>.
- [13] Al Musawi AF, Roy S, Ghosh P. Examining indicators of complex network vulnerability across diverse attack scenarios. *Sci Rep* Oct. 2023;13(1):18208. <https://doi.org/10.1038/s41598-023-45218-9>.
- [14] Callaway DS, Newman MEJ, Strogatz SH, Watts DJ. Network robustness and fragility: percolation on random graphs. *Phys Rev Lett* 2000;85(25):5468–71. <https://doi.org/10.1103/PhysRevLett.85.5468>.
- [15] Valdez LD, et al. Cascading failures in complex networks. *J. Complex Netw* 2020;8(2):1–23. <https://doi.org/10.1093/COMNET/CNAA013>. Oxford University Press.
- [16] Braunstein A, Dall'Asta L, Semerjian G, Zdeborová L. Network dismantling. *Proc Natl Acad Sci U S A* Nov. 2016;113(44):12368–73. <https://doi.org/10.1073/pnas.1605083113>.
- [17] Ren XL, Gleinig N, Helbing D, Antulov-Fantulin N. Generalized network dismantling. *Proc Natl Acad Sci U S A* 2019;116(14):6554–9. <https://doi.org/10.1073/pnas.1806108116>.
- [18] Lei M, Cheong KH. Node influence ranking in complex networks: a local structure entropy approach. *Chaos Solit Fract* 2022;160:112136. <https://doi.org/10.1016/j.chaos.2022.112136>.
- [19] Wandelt S, Lin W, Sun X, Zanin M. From random failures to targeted attacks in network dismantling. *Reliab Eng Syst Saf* Feb. 2022;218:108146. <https://doi.org/10.1016/j.res.2021.108146>.
- [20] Feng Z, Song H, Qi X. A novel algorithm for the generalized network dismantling problem based on dynamic programming. *Chaos Solit Fract* 2024;180, no. January: 114585. <https://doi.org/10.1016/j.chaos.2024.114585>.
- [21] Di Summa M, Grosso A, Locatelli M. Branch and cut algorithms for detecting critical nodes in undirected graphs. *Comput Optim Appl* 2012;53(3):649–80. <https://doi.org/10.1007/s10589-012-9458-y>.
- [22] Holme P, Kim BJ, Yoon CN, Han SK. Attack vulnerability of complex networks. *Phys Rev E - Stat Physics, Plasmas, Fluids, Relat Interdiscip Top* 2002;65(5):14. <https://doi.org/10.1103/PhysRevE.65.056109>.
- [23] Huang W, et al. Node importance identification of unweighted urban rail transit network: an adjacency information entropy based approach. *Reliab Eng Syst Saf* 2024;242(May 2023):109766. <https://doi.org/10.1016/j.res.2023.109766>.
- [24] Guo H, Wang S, Yan X, Zhang K. Node importance evaluation method of complex network based on the fusion gravity model. *Chaos Solit Fract* 2024;183, no. May: 114924. <https://doi.org/10.1016/j.chaos.2024.114924>.
- [25] Geisberger R, Sanders P, Schultes D. Better approximation of betweenness centrality. In: 2008 Proceedings of the Tenth Workshop on Algorithm Engineering and Experiments (ALENEX); 2008. p. 90–100.
- [26] Rui X, Meng F, Chai Y, Wang Z, Yu PS. Dismantling networks by skeleton extraction and greedy tree breaking. *IEEE Access* 2021;9:84922–31. <https://doi.org/10.1109/ACCESS.2021.3086099>.
- [27] Lozano M, García-Martínez C, Rodríguez FJ, Trujillo HM. Optimizing network attacks by artificial bee colony. *Inf Sci (Ny)* Jan. 2017;377:30–50. <https://doi.org/10.1016/j.ins.2016.10.014>.
- [28] Lin W, Wandelt S, Sun X. Efficient network dismantling through genetic algorithms. *Soft Comput Mar.* 2022;26(6):3107–25. <https://doi.org/10.1007/s00500-021-06475-w>.
- [29] Lou Y, Wu R, Li J, Wang L, Li X, Chen G. A learning convolutional neural network approach for network robustness prediction. *IEEE Trans Cybern* 2023;53(7): 4531–44. <https://doi.org/10.1109/TCYB.2022.3207878>.
- [30] Jiang W, Fan T, Li C, Zhang C, Zhang T, Luo Z Fu. Comprehensive analysis of network robustness evaluation based on convolutional neural networks with spatial pyramid pooling. *Chaos Solit Fract* 2024;184, no. May:115023. <https://doi.org/10.1016/j.chaos.2024.115023>.
- [31] Fan C, Zeng L, Sun Y, Liu YY. Finding key players in complex networks through deep reinforcement learning. *Nat Mach Intell Jun.* 2020;2(6):317–24. <https://doi.org/10.1038/s42256-020-0177-2>.
- [32] Fan C, Sun Y, Zeng L, Liu Y-Y, Chen M, Liu Z. Dismantle large networks through deep reinforcement learning. In: ICLR representation learning on graphs and manifolds workshop; 2019. p. 1–5.
- [33] Fan C, Zeng L, Feng Y, Cheng G, Huang J, Liu Z. A novel learning-based approach for efficient dismantling of networks. *Int J Mach Learn Cybern Sep.* 2020;11(9): 2101–11. <https://doi.org/10.1007/s13042-020-01104-8>.
- [34] Yan D, Xie W, Zhang Y, He Q, Yang Y. Hypernetwork dismantling via deep reinforcement learning. *IEEE Trans Netw Sci Eng* 2022;9(5):3302–15. <https://doi.org/10.1109/TNSE.2022.3174163>.
- [35] Xing Y, Shu H, Kang F. Peer remove: an adaptive node removal strategy for P2P botnet based on deep reinforcement learning. *Comput Secur May* 2023;128: 103129. <https://doi.org/10.1016/j.cose.2023.103129>.
- [36] Arulkumaran K, Deisenroth MP, Brundage M, Bharath AA. Deep reinforcement learning: A brief survey. *IEEE Signal Process Mag Nov.* 01, 2017;34(6):26–38. <https://doi.org/10.1109/MSP.2017.2743240>. Institute of Electrical and Electronics Engineers Inc.
- [37] Montavon G, Samek W, Müller KR. Methods for interpreting and understanding deep neural networks. *Digital Signal Process: Rev J Feb.* 01, 2018;73:1–15. <https://doi.org/10.1016/j.dsp.2017.10.011>. Elsevier Inc.
- [38] Grassia M, De Domenico M, Mangioni G. Machine learning dismantling and early-warning signals of disintegration in complex systems. *Nat Commun Dec.* 2021;12(1):5190. <https://doi.org/10.1038/s41467-021-25485-8>.
- [39] Grassia M, Mangioni G. CoreGDM: Geometric deep learning network decycling and dismantling. In: Springer proceedings in complexity. Springer Science and Business Media B.V.; 2023. p. 86–94. https://doi.org/10.1007/978-3-031-28276-8_8.
- [40] Zhang J, Wang B. Dismantling complex networks by a neural model trained from tiny networks. In: International conference on information and knowledge management, proceedings. Association for Computing Machinery; Oct. 2022. p. 2559–68. <https://doi.org/10.1145/3511808.3557290>.
- [41] Liu Q, Wang B. Neural extraction of multiscale essential structure for network dismantling. *Neural Netw Oct.* 2022;154:99–108. <https://doi.org/10.1016/j.neunet.2022.07.015>.
- [42] Dike HU, Zhou Y, Deveerasetty KK, Wu Q. Unsupervised learning based on artificial neural network: a review. 2018 IEEE Int. Conf Cyborg Bionic Syst CBS 2018 2018;(October):322–7. <https://doi.org/10.1109/CBS.2018.8612259>.
- [43] Zhang J, Wang B. Encoding node diffusion competence and role significance for network dismantling. In: *ACM web conference 2023 - proceedings of the world wide web conference*, WWW 2023; 2023. p. 111–21. <https://doi.org/10.1145/3543507.3583233>.
- [44] Henderson K, Gallagher B, Eliassi-Rad T, Tong H, Basu S. Rolx: structural role extraction & mining in large graphs. In: Proceedings of the 18th ACM SIGKDD international conference on Knowledge discovery and data mining; 2012. p. 1231–9.
- [45] Zhang S, Tong H, Xu J, Maciejewski R. Graph convolutional networks: a comprehensive review. *Comput Soc Networks Dec.* 2019;6(1):1–23. <https://doi.org/10.1186/s40649-019-0069-y>.
- [46] Artimo O, Grassia M, De Domenico M, Gleeson JP. Robustness and resilience of complex networks. *Nat Rev Phys* 2024:1–18.
- [47] Veličković P, Cucurull G, Casanova A, Romero A, Liò P, Bengio Y. Graph attention networks. *Stat Oct.* 2017;1050(20):10–48550 [Online]. Available, <http://arxiv.org/abs/1710.10903>.
- [48] Kafadar K, Koehler JR, Venables WN, Ripley BD. Modern applied statistics with S-plus. *Am Stat* 1999;53(1):86. <https://doi.org/10.2307/2685660>.
- [49] Molloy M, Reed B. A critical point for random graphs with a given degree sequence. *Random Struct algorithms* 1995;6(2–3):161–80.
- [50] Li HJ, Wang L, Bu Z, Cao J, Shi Y. Measuring the network vulnerability based on Markov criticality. *ACM Trans Knowl Discov Data Apr.* 2021;16(2):1–24. <https://doi.org/10.1145/3464390>.
- [51] Schneider CM, Moreira AA, Andrade JS, Havlin S, Herrmann HJ. Mitigation of malicious attacks on networks. *Proc Natl Acad Sci U S A* 2011;108(10):3838–41. <https://doi.org/10.1073/pnas.1009440108>.
- [52] He K, Zhang X, Ren S, Sun J. Deep residual learning for image recognition. In: Proceedings of the IEEE computer society conference on computer vision and pattern recognition; Dec. 2016. p. 770–8. <https://doi.org/10.1109/CVPR.2016.90>.
- [53] Xiao B, et al. PAM-DenseNet: a deep convolutional neural network for computer-aided COVID-19 diagnosis. *IEEE Trans Cybern* 2022;52(11):12163–74. <https://doi.org/10.1109/TCYB.2020.3042837>.
- [54] Zagoruyko S, Komodakis N. Wide residual networks. *arXiv Prepr May* 2016:1–15. <https://arxiv.org/abs/1605.07146>. [Online]. Available, <https://arxiv.org/abs/1605.07146>.
- [55] Taud H, Mas JF. Multilayer perceptron (MLP). 2018. p. 451–5. https://doi.org/10.1007/978-3-319-60801-3_27.
- [56] Wang L, et al. A survey on large language model based autonomous agents. *Front Comput Sci Aug.* 2024;18(6):186345. <https://doi.org/10.1007/s11704-024-40231-1>.
- [57] Lipowski A, Lipowska D. Roulette-wheel selection via stochastic acceptance. *Phys A Stat Mech its Appl Mar.* 2012;391(6):2193–6. <https://doi.org/10.1016/j.physa.2011.12.004>.
- [58] Watts DJ, Strogatz SH. Collective dynamics of ‘small-world’ networks. *Nature* 1998;393(6684):440–2. <https://doi.org/10.1038/30918>.
- [59] Barabási A L R. Emergence of scaling in random networks. *Science (80-)* 1999; 286(5439):509–12.
- [60] Wandelt S, Wandelt S, Shi X, Sun X, Zanin M. Community detection boosts network dismantling on real-world networks. *IEEE Access* 2020;8:111954–65. <https://doi.org/10.1109/ACCESS.2020.3002807>.
- [61] Shang WL, Chen Y, Song C, Ochieng WY. Robustness analysis of urban road networks from topological and operational perspectives. *Math Probl Eng* 2020; 2020(1):5875803. <https://doi.org/10.1155/2020/5875803>.
- [62] Olaf J, Möhring RH, Schulz AS, Moses NES. System-optimal routing of traffic flows with user constraints in networks with congestion. *Oper Res* 2005;53(4):600–16.
- [63] Zhang J, Pourazarm S, Cassandras CG, Paschalidis IC. The price of anarchy in transportation networks by estimating user cost functions from actual traffic data. In: 2016 IEEE 55th conference on decision and control (cdc); 2016. p. 789–94.

- [64] Kunegis J. Konect: the koblenz network collection. In: *Proceedings of the 22nd international conference on world wide web*; 2013. p. 1343–50.
- [65] Lancichinetti A, Fortunato S, Radicchi F. Benchmark graphs for testing community detection algorithms. *Phys Rev E - Stat Nonlinear, Soft Matter Phys* 2008;78(4): 1–6. <https://doi.org/10.1103/PhysRevE.78.046110>.
- [66] Yu EY, Fu Y, Chen X, Xie M, Chen DB. Identifying critical nodes in temporal networks by network embedding. *Sci Rep* 2020;10(1):1–8. <https://doi.org/10.1038/s41598-020-69379-z>.